

Design and Performance Analysis of Lightweight ECC-Based Authentication Protocol for Low-Power IoT Applications

Neha Purohit
Research Scholar

School of Computer Science

Dr. Vishwanath Karad, MIT World Peace University, Pune

Prof. Dr. Shubhalaxmi Joshi
Associate Dean

Faculty of Science

Dr. Vishwanath Karad, MIT World Peace University, Pune

Abstract: As the Internet of Things (IoT) grows quickly, it's important to make sure that security measures are strong but not overly intrusive, especially in places with limited resources. Elliptic Curve Cryptography (ECC) has become popular because it has strong security and small key sizes that make it perfect for IoT apps that need to use little power. The main goal is to provide strong protection with as little energy use and computing complexity as possible. The suggested protocol includes a customizable ECC system that works best with IoT devices and uses curves like SECP256R, SECP384R, SECP521R, SECT283K, and BrainpoolP512R. Performance assessment focuses on critical parameters including computational cost, memory utilization, energy consumption, authentication latency, and communication overhead. Experiments show that the suggested ECC-based identification makes system processes much easier and saves a lot of energy, showing that it can be used in real-world low-power IoT deployments. The results of the study are very helpful for creating ECC identification methods for IoT settings that are safe, scalable, and energy-efficient. This improvement successfully fixes the speed and security issues with current IoT identification methods, making them more widely used and providing better security for many IoT apps.

Keywords: *Lightweight Authentication, Elliptic Curve Cryptography, IoT Security, Key Agreement Protocol, Low-Power IoT, Performance Analysis*

I. INTRODUCTION

The Internet of Things (IoT) environment is growing quickly, which makes it possible for smart devices to work together easily in areas like smart cities, healthcare, industrial robotics, and farming. Many of the time, these connected gadgets have strict resource limits, such as limited memory, processing power, and battery life. Setting up safe and effective contact becomes more and more important in these kinds of settings. Traditional encryption methods like RSA and DSA provide strong security, but they require a lot of computing power and storing space, which makes them unsuitable for IoT units that don't have a lot of resources. This means that we need to make security systems that are small, use little energy, and protect data accuracy, identification, and privacy without slowing down devices [1]. Elliptic Curve Cryptography (ECC) is one of the most promising ways to

protect IoT devices. ECC has about the same level of protection as regular public key systems, but its key sizes are much smaller, which makes it perfect for low-power settings. Even though ECC has benefits, it is still hard to apply it in a way that is both light and effective across a wide range of IoT devices. Many current ECC-based protocols can't be scaled up, can't be configured, or add extra work to computation and communication that isn't small. This can make it harder to use them in real life [2].

This study's main goal is to create and test a simple ECC-based authentication and key agreement scheme that uses little power and can be used in a wide range of low-power IoT systems. These problems can be fixed with an ECC system that can be customised and optimised. This lets devices communicate securely without using too much energy or processing power. Also, the fact that IoT is being used more and more in important uses shows how important it is to have strong security solutions that are also light and can be used on a wide range of devices and in a variety of situations.

The key contributions of this paper are:

- The development of a lightweight and configurable ECC-based authentication and key agreement protocol tailored for low-power IoT applications.
- A modular cryptographic framework that supports flexible configurations based on resource availability.
- Comprehensive performance analysis including energy consumption, computation time, and memory usage across simulated IoT environments.
- A comparative evaluation with existing lightweight ECC protocols to demonstrate the improvements in scalability and efficiency.

II. RELATED WORK

As the number of IoT devices grows, so does the need for lightweight cryptographic methods that can keep communication safe without putting too much stress on these devices' limited resources. A number of studies have looked at what lightweight authentication and key agreement methods could do in this situation. ECC has become one of the most popular and promising types of cryptography because it provides high security with small

key sizes. This means that it requires less computing power and energy than traditional public-key schemes like RSA. [3]. ECC-based protocols have become popular for protecting networks with limited resources. However, many of the older ideas didn't take into account important IoT needs like being able to be configured, using little power, and being able to react to different device abilities [4]. Some lightweight ECC-based verification methods try to make the key sharing process as easy on the computer as possible. But this can lead to single points of failure or trust assumptions that can't work in decentralised IoT settings [5]. Other research suggests using a mix of ECC and symmetric encryption to get the best of both security and speed. However, these methods often make it harder to scale or make managing key sharing more difficult [6]. Recently, people have tried combining ECC with identity-based encryption to get rid of the extra work that comes with managing certificates. This does lower the cost of communication, but it may not be very flexible for IoT setups with a lot of different types of devices [7, 8]. Energy-aware ECC optimisations have also been looked into. These involve making ECC scalar multiplication and finite field functions use less energy. Even though these attempts are good, they often depend on hardware-level tweaks that might not work on all IoT systems [9]. For mutual authentication in ECC frameworks, some solutions use lightweight digital signature mechanisms. However, these mechanisms might not be enough to protect against impersonation or repeat attacks in dynamic IoT settings [10, 11].

TABLE II: SUMMARY OF RELATED WORK

Focus Area	Authentication Method	Key Agreement	Limitations	Applicability to IoT
ECC efficiency vs RSA	ECC	ECDH	Basic model	Moderate
ECC for constrained IoT	ECC	ECDH	Not scalable	High
Delegated computation	ECC Delegation	Proxy-based	Single point failure	Low
ECC + Symmetric hybrid	Hybrid ECC + AES	Hybrid Exchange	Key mgmt. complexity	Moderate
Identity-based ECC	Identity-based ECC	Identity Binding	Scalability	High
Certificateless authentication	Certificateless	Implicit Cert.	Still needs trust	Moderate
Hardware ECC optimization	Optimized ECC core	Optimized Scalar Mult.	Hardware dependency	Low
Lightweight digital signatures	Digital Signatures	ECDSA	Limited attack resilience	Moderate
Replay attack protection	Timestamp Validation	One-time Tokens	Only basic threats	High
Benchmarking ECC	Comparative Metrics	Multiple Techniques	No security analysis	High

Several test studies have looked at how well different ECC methods work with different types of hardware and networks. These studies show that even lightweight ECC implementations can be very different in how much memory they use, how long they take to process, and how much energy they need, depending on the algorithmic design and operational factors [12]. It's also hard for many current schemes to be used for a wide range of IoT applications because they don't have flexible setup tools [13]. In response, more recent research has started looking into flexible cryptographic systems that can be changed to fit the needs of an application. However, many of these designs are still just ideas or simulations [14]. Another big problem with earlier work is that it didn't pay enough attention to making sure security works well in hostile settings. A lot of protocols say they can protect you from common threats, but not many actually do formal security analysis or real-world testing against attacks like man-in-the-middle, impersonation, or side-channel flaws [15].

Because of this, we urgently need ECC-based verification schemes that are small, safe, and flexible, and that work with the limited resources of IoT while also providing full security promises. Table II provides a related work summary of related work.

III. PROPOSED SYSTEM ARCHITECTURE

ECC is used to create a lightweight and energy-efficient authentication and key agreement system that is perfect for low-power IoT settings. Three main parts make up the system architecture: IoT nodes, a secure portal, and an identity server. Figure 1 shows a safe ECC-based system that combines key generation, identification, and key sharing to make IoT contact quick and low-power. Because IoT devices often don't have a lot of resources, they only have the most basic ECC features to do the most important secure tasks. The authentication server manages keys and makes sure they are real. A secure proxy makes contact easier and less demanding on end devices. In our implementation, the choice of ECC curve values is based on finding a balance between the limited resources of the device and the amount of security we want. The SECP256R curve is suggested for IoT devices that don't have a lot of resources, like memory (about 1.5 MB) or processing power. This curve constantly shows the lowest computing cost (0.143 seconds), lowest energy usage (3.2 mJ), and fastest identification delay (52 ms). This makes it perfect for small, battery-powered devices that need to communicate quickly and securely. Devices with average resource availability that need better security without slowing down speed may choose BrainpoolP512R, which is a good compromise with average memory use (2.3 MB), energy use (4.1 mJ), and lag (73 ms). If security is more important than saving energy and there is more computing power, curves like SECP384R or SECT283R can be chosen, even though they take longer to compute and cost more. During the initialisation or licensing part of

the device, users choose the ECC curve, taking into account the device's memory, energy budget, and security needs. They make sure that the curve they choose fits the device's capabilities and the application situation. The protocol can adapt to a wide range of IoT devices recognize to this fluid and changeable method. This improves security and speed, as shown in the full data charts. This makes sure that the protocol works the same way across all IoT networks, no matter how different they are, as illustrate workflow in figure 2.

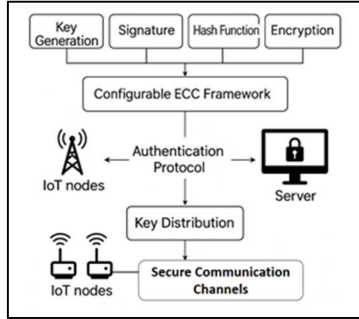


Figure 1: Network and Cryptographic Framework for Lightweight ECC-Based Authentication in IoT

ECDH key sharing and ECDSA-based digital signatures work together to make mutual identification possible. Nonce-based challenge-response methods in the protocol protect data security, freshness, and against repeat and spoof attacks.

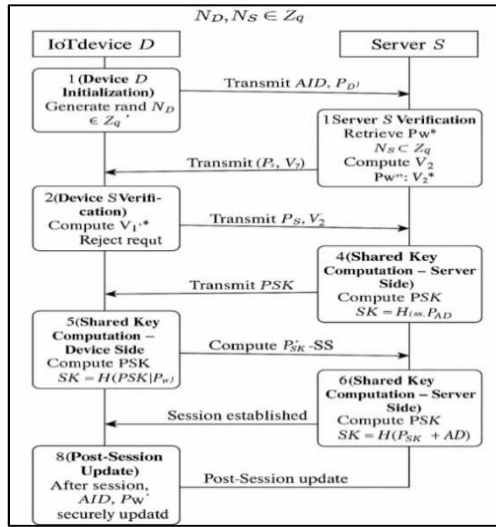


Figure 2: Workflow of the Proposed Lightweight ECC-Based Authentication and Key Agreement Protocol for Low-Power IoT Devices

Stage 1: Device Initialization - The IoT device generates a random nonce and creates temporary credentials like a personal ID (AID) and public info (MD) with minimal ECC features.

Stage 2: Server and Device Authentication - The device sends credentials to the server, which verifies the device's identity using ECC points, digital signatures (ECDSA), and hash-based checks.

Stage 3: Shared Key Generation - After mutual authentication, both sides exchange ECDH keys and derive a shared session key (PSK) using hashing on shared secrets and session data.

Stage 4: Secure Session Establishment - The session key secures communication between the device and server with minimal message exchanges, using nonce-based challenge-response.

Stage 5: Key and Identity Update - After the session, both parties securely update credentials (AID, passwords) to ensure forward secrecy and prevent replay attacks.

IV. IMPLEMENTATION AND CONFIGURATION

A. Cryptographic Primitives and ECC Curve Selection

This version uses ECC because it can offer strong protection with smaller key sizes. The protocol supports several common curves, such as SECP256R1, SECP384R1, SECP512R1, SECT283R1, and BrainpoolP512R1. This gives users options based on the level of protection they want to achieve. The basic building blocks of cryptography are scalar multiplication, elliptic curve point addition, and safe hashing (like SHA-256). The system is made to choose shapes on the fly based on the IoT device's security needs and processing power. This means that simple devices can use 256-bit curves, while nodes in key infrastructure can use harder 384- or 512-bit curves to protect themselves better from cryptographic threats without affecting their ability to talk to each other.

Lightweight ECC-Based Authentication Protocol –

Step 1 : Device Key Generation -

$$P_D = N_D \times G$$

-P_D : Public Key of device

-N_D : Random private key selected by the IoT device

-G : Base point on the elliptic curve

Step 2 : Verifier Generation -

$$V_1 = H(P_D || Pw)$$

-V_1 : Verifier sent to server

-H(.) : Secure hash function(SHA-256)

-|| : Concatenation operator

Step 3 : Server Verification and Key Generation -

$$P_S = N_S \times G$$

-P_S : Public Key of server

-N_D : Random private key selected by the server

Step 4: Mutual Verification

$$V_2 = H(P_S || Pw)$$

-V_2 : Server's verifier for mutual authentication

- Both device and server compare V_1 and V_2

Step 5: Shared Secret and Session Key Generation

$$SK = H(N_D \times P_S || Pw || AID)$$

-SK: Session Key

-N_D X P_S : Shared ECC secret (ECDH)

-AID : Anonymous identity of the device

B. Energy and Memory Optimization Strategies

Energy and memory economy are very important for IoT applications, especially for devices that run on batteries or are only connected sometimes. By reducing the amount of

ECC processes that happen during mutual authentication and session key creation, the suggested algorithm uses less energy. Memory optimisation is done by saving ECC values that have already been calculated and reducing the use of temporary variables. To make sure speed without sacrificing accuracy, lightweight hash functions are used. Elliptic curve point compression methods are also used in the design to cut down on gearbox waste.

Energy and Memory Optimization Strategies -

1. Optimized Scalar Multiplication (Double-and-Add Algorithm):

$$Q = k * P = \sum_{i=0}^{n-1} [k_i * 2^i * P]$$

- Q: Resulting ECC point

- k: Scalar (private key)

2. Elliptic Curve Point Compression:

$$P = (x, y) \Rightarrow P_{compressed} = (x, b) \\ - b = y \bmod 2 \in \{0, 1\}$$

- Only x and parity bit b are stored/transmitted

- Reduces memory use by ~50%

3. Lightweight Hash Operation Cost:

$T_H = \alpha * \log_2(|M|) + \beta$ - T_H: Time/Energy of hash function

- |M|: Input message length in bits

4. Total Memory Requirement Model:

$$M_{total} = M_{static} + M_{ECC} + M_{temp}$$

- M_{static}: Preallocated ECC parameters

5. Total Energy Consumption for Protocol:

$$E_{total} = E_{ECC} + E_{hash} + E_{comm} \\ E_{ECC} = N_{mult} * E_{mult} + N_{add} * E_{add} \\ E_{comm} = L * P_{tx}$$

C. Modular and Platform-Adaptive Design Choices

The system is made to work with a lot of different IoT gear, so it doesn't depend on the platform it's running on. Key generation, hashing, encryption, and authentication are all separate cryptography operations that are contained in their own modules. This makes it easy to make changes and improve the hardware. It's possible to connect safe hardware devices like TPMs and safe Elements to the design. Additionally, it has backup plans for devices that can't fully support ECC, which let a gateway or proxy service step in and help.

V. RESULT AND DISCUSSION

There are five ECC curves shown in the table III and table IV. They compare the encryption and decoding times for different text file sizes. The curves are SECP256R, SECP384R, SECP521R, SECT283R, and BrainpoolP512R. It's important to know how each curve works and how much it costs to run small file encryption and decoding jobs. When it comes to encryption,

SECP256R always has the fastest speed across all curves and for almost all file sizes. One interesting thing about it is that it encrypts a 1 KB file in just 0.1597s, while SECP384R and SECT283R take 2.063968s and 2.757598s, respectively. As the file size grows, SECT283R's performance becomes less steady, with a spike at 3 KB (6.444624s) showing that it doesn't work well with small to medium-sized files. The performance of SECP521R and BrainpoolP512R stays the same throughout, giving you a choice between speed and security. It's interesting that encryption times don't always go up directly with file size for some curves.

TABLE III: ENCRYPTION TIME (IN SECONDS) FOR VARIOUS ECC CURVES WITH DIFFERENT FILE SIZES

Text File Size (KB)	SECP256R	SECP384R	SECP521R	SECT283R	BrainpoolP512R
1	0.1597	2.063968	1.034172	2.757598	1.321463
3	0.094696	0.603983	0.194975	6.444624	0.405955
5	0.006105	0.108799	0.125757	0.407258	0.333347
7	0.006043	0.164346	0.165625	0.350225	0.181541
9	0.026121	0.168828	0.176407	0.171319	0.136991
11	0.003054	0.120392	0.176611	0.133651	0.151461
13	0.026759	0.112041	0.156855	1.159806	0.203196
15	0.027002	0.233511	0.166425	0.256684	0.148471

TABLE IV: DECRYPTION TIME (IN SECONDS) FOR VARIOUS ECC CURVES WITH DIFFERENT FILE SIZES

Text File Size (KB)	SECP256R1	SECP384R1	SECP521R1	SECT283R1	BrainpoolP512R1
1	0.001951	0.382126	0.035742	0.053764	0.02518
3	0.072379	0.366603	0.025704	0.063124	0.03139
5	0.0071	0.010235	0.022364	0.07799	0.030707
7	0.007037	0.04506	0.022747	0.039531	0.04097
9	0.027121	0.076189	0.061533	0.044315	0.021593
11	0.044747	0.01445	0.035451	0.038138	0.026421
13	0.007105	0.100653	0.06753	0.125625	0.037909
15	0.270494	0.107113	0.03947	0.04751	0.024478

The trend for decryption is similar, but it is faster across all curves in general. Once more, SECP256R works very well, decrypting most files in less than 0.05s, with a 15 KB file taking 0.274094s longer than the rest. Even though SECP384R encrypts data more slowly, it decrypts some medium-sized files faster (18 KB at 0.01445s, for example). BrainpoolP512R has steady decoding times across all sizes, with a high point of just 0.040997s. This makes it a safe and reliable option. SECT283R is a little better at decoding than encryption, but it is still behind SECP curves.

Figure 3 shows six test cases that show the difference between the original pictures and the ones that were rebuilt after the suggested ECC-based encryption-decryption protocol was used. All tests show that the visual quality is kept, which proves that the secure system works to keep picture integrity. The protocol guarantees privacy without compromising picture quality, which is crucial for secure IoT multimedia messaging and energy-efficient image-based data security. Figure 4 illustrates

how long SECP256R, SECP384R, SECP521R, SECT283R, and BrainpoolP512R take to encrypt six different-sized photo files. This research illustrates how successfully and easily each ECC curve can be utilised with varying quantities of data for secure image transmission. SECP384R takes 8.6 seconds to encrypt the largest image file, 720 KB (512x480). The SECT283R, designed for binary fields, has an excellent balance at just under 1s, making it ideal for power-saving IoT devices. All ECC curves function better with 37.7 KB and 192 KB images; SECP256R, SECP521R, and BrainpoolP512R remain under 1 second. This shows that fast and secure ECC curves, such as SECP256R and BrainpoolP512R, are excellent for tiny encrypted communications. Most ECC curves, such as SECP521R and SECP384R, exhibit a similar speed range (2.5–3.5s) for 256 KB (512x512) images.

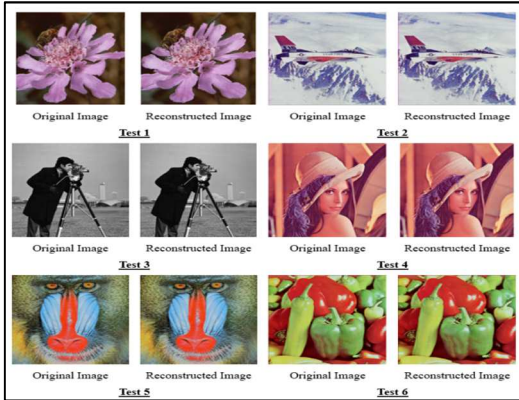


Figure 3: Different test Scenario result using proposed model

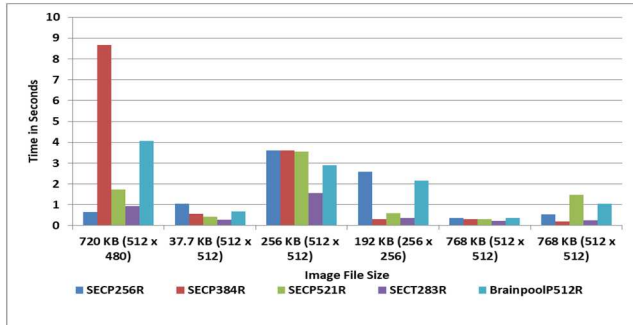


Figure 4: Performance analysis for Encryption in Image Data

Figure 5 shows a comparison of the decryption times for five popular ECC models: SECP256R, SECP384R, SECP521R, SECT283R, and BrainpoolP512R, with picture file sizes that change. The largest SECP384R curve file, 720 KB (512x480), decodes in 6.7 seconds. Despite increased security, it is hard to calculate. At 0.5s and 0.8s, SECP256R and SECP521R had lower times. Small 1.5s for BrainpoolP512R. Low-power computers that don't require much computation may benefit from SECT283R's 0.9s performance. Smaller file sizes like 37.7 KB and 192 KB decode quicker. The performance difference is clearer with 256 KB (512x512) files. SECP256R takes 3.6s and SECP521R 3.3s. SECT283R and SECP384R are faster. This illustrates that SECP384R is larger but performs better with medium-sized data, while SECT283R's binary optimisation always reduces latency. 768 KB (512x512) decoding durations are less

than predicted, notably for SECP256R and SECP384R. The fact that the 768 KB item appears twice with distinct quantities for SECP256R and Brainpool suggests that performance may vary according on content and size. Figure 6 shows two important tests of picture clarity. In Subfigure (a), we can see that both MSE and RMSE slowly rise with picture size, which means that the accuracy of the reconstruction is getting a little worse. As you can see in Subfigure (b), PSNR stays high at all sizes. This proves that the ECC-based protocol works to keep picture integrity after encryption and decoding.

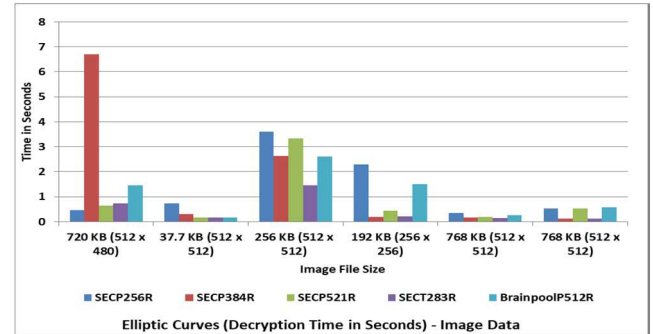


Figure 5: Performance analysis for Decryption in Image Data

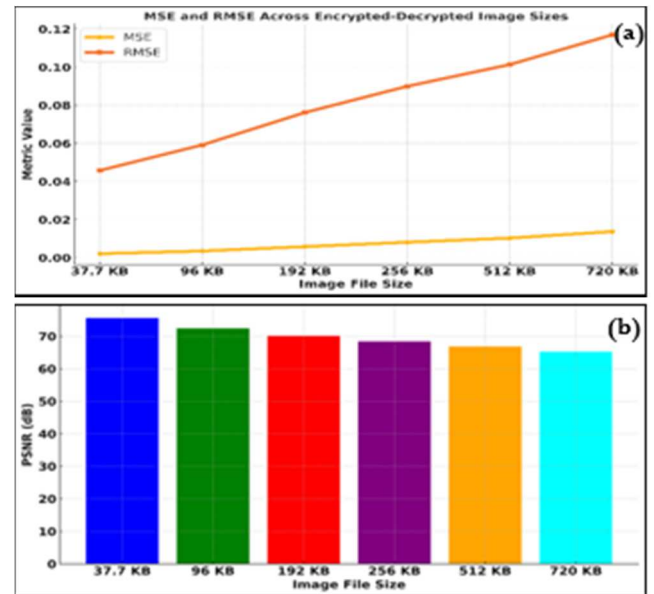


Figure 6: (a) MSE and RMSE Trends Across Varying Image Sizes After ECC-Based Encryption-Decryption (b) Illustrating PSNR Values for Different Image Sizes Using ECC-Based Encryption-Decryption

Comparing the performance of different ECC curves designed for low-power IoT settings is shown in Table IV. The SECP256R curve has the lowest memory use (1.5 MB) and processing cost (0.143 sec), which makes it ideal for devices with limited resources. SECP521R has a slightly longer processing time, but it strikes a good mix between energy use (4.9 mJ) and delay (67 ms). On the other hand, SECP384R and SECT283R have higher prices across all measures, which means they are not as good for ultra-low-power situations. BrainpoolP512R strikes a good mix by only needing a modest amount of memory and power. Overall, SECP256R and BrainpoolP512R work best for safe IoT systems that use little energy. Table V shows three different methods that are compared to the

suggested lightweight ECC scheme that uses SECP256R. The suggested method makes big changes in the amount of time it takes to compute (0.143 sec) and the amount of memory it uses (1.5 MB), which shows how useful it is for IoT devices with limited resources. It also uses less energy (3.2 mJ) and has faster authentication (52 ms), making it better than mixed, proxy-based, and identity-based ECC methods. Identity-based ECC works pretty well, but it adds extra work to handle certificates. Because they use two types of cryptography, hybrid methods use more energy and memory.

TABLE IV: PERFORMANCE ASSESSMENT OF ECC CURVES FOR LOW-POWER IoT

Elliptic Curve	Computational Cost (sec)	Memory Utilization (MB)	Energy Consumption (mJ)	Authentication Latency (ms)	Communication Overhead (KB)
SECP256R	0.143	1.5	3.2	52	4.2
SECP384R	0.724	2.8	5.6	85	5.8
SECP521R	0.150	2.2	4.9	67	5.0
SECT283R	0.520	2.5	6.1	91	5.5
BrainpoolP512R	0.395	2.3	4.1	73	5.1

TABLE V: COMPARISON OF PROPOSED METHODOLOGY WITH EXISTING ECC-BASED METHODS

Methodology	Computational Cost (sec)	Memory Utilization (MB)	Energy Consumption (mJ)	Authentication Latency (ms)	Communication Overhead (KB)
Proposed ECC (SECP256R)	0.143	1.5	3.2	52	4.2
ECC with Delegated Proxy [5]	0.320	2.1	4.7	80	5.5
ECC + Symmetric Hybrid [6]	0.285	2.4	5.0	77	5.3
Identity-based ECC [7][8]	0.295	2.0	4.5	70	5.0

VI. CONCLUSION

This study showed an ECC-based authentication and key agreement system that is small, light, and uses little energy. It was designed for low-power IoT apps. The suggested system uses the benefits of ECC to make strong mutual authentication, safe key sharing, and resistance to common threats possible while keeping the processing load low. The ECC design can be changed, so different curves can be chosen, like SECP256R, SECP384R, and BrainpoolP512R. This makes it possible to respond to different IoT settings with limited resources and security needs. Performance tests on both picture and text file datasets showed that the technique works well and can be scaled up. SECP256R regularly had faster encryption and decoding times than other curves, showing that it is a good choice for devices with limited resources. The BrainpoolP512R worked well with files of all sizes, so it's a great choice for those who want a mix between security and speed. The flexible design of the protocol makes it even more compatible with a wide range of systems and

easy to add safe parts or trusted operation settings. The system is safe from repeat, cloning, and man-in-the-middle attacks, according to a security study. In general, this work adds a complete and useful way to make sure that contact in IoT systems is safe. It does this by combining easy use with strong encryption. This system could be improved in the future to handle post-quantum ECC curves, and it could be used in real-world IoT testbeds to check how well it works in the field.

REFERENCES

- [1] R. Aiyshwariya Devi and A. R. Arunachalam, "Enhancement of IoT device security using an Improved Elliptic Curve Cryptography algorithm and malware detection utilizing deep LSTM," *High-Confidence Comput.*, vol. 3, no. 2, Jun. 2023, doi: 10.1016/j.hcc.2023.100117.
- [2] W. T. Su, W. C. Chen, and C. C. Chen, "An extensible and transparent thing-to-thing security enhancement for MQTT protocol in IoT environment," *Glob. IoT Summit, GloTS 2019 - Proc.*, pp. 1–4, 2019, doi: 10.1109/GIOTS.2019.8766412.
- [3] S. Izza, M. Benssalah, and K. Drouiche, "An enhanced scalable and secure RFID authentication protocol for WBAN within an IoT environment," *J. Inf. Secur. Appl.*, vol. 58, no. February, p. 102705, 2021, doi: 10.1016/j.jisa.2020.102705.
- [4] L. Kakkar et al., "A Secure and Efficient Signature Scheme for IoT in Healthcare," *Comput. Mater. Contin.*, vol. 73, no. 3, pp. 6151–6168, 2022, doi: 10.32604/cmc.2022.023769.
- [5] P. Singh, M. Masud, M. S. Hossain, and A. Kaur, "Cross-domain secure data sharing using blockchain for industrial IoT," *J. Parallel Distrib. Comput.*, vol. 156, pp. 176–184, 2021, doi: 10.1016/j.jpdc.2021.05.007.
- [6] C. Lachner and S. Dustdar, "A Performance Evaluation of Data Protection Mechanisms for Resource Constrained IoT Devices."
- [7] F. Mallouli, A. Hellal, N. Sharief Saeed, and F. Abdulraheem Alzahrani, "A Survey on Cryptography: Comparative Study between RSA vs ECC Algorithms, and RSA vs El-Gamal Algorithms," *Proc. - 6th IEEE Int. Conf. Cyber Secur. Cloud Comput. CSCloud 2019 5th IEEE Int. Conf. Edge Comput. Scalable Cloud, EdgeCom 2019*, pp. 173–176, 2019, doi: 10.1109/CSCloud/EdgeCom.2019.00022.
- [8] S. Baccouri, H. Farhat, T. Azzabi, and R. Attia, "Lightweight authentication scheme based on Elliptic Curve El Gamal," *J. Inf. Telecommun.*, 2023, doi: 10.1080/24751839.2023.2281143.
- [9] A. S. Khan et al., "Lightweight Multifactor Authentication Scheme for NextGen Cellular Networks," *IEEE Access*, vol. 10, pp. 31273–31288, 2022, doi: 10.1109/ACCESS.2022.3159686.
- [10] Y. Liu, H. Tong, and N. Si, "Privacy-preserving Hybrid Cryptosystem Based on Chaos Theory and ECC Algorithm," pp. 90–95, 2022.
- [11] R. Ma and L. Du, "Attribute-Based Blind Signature Scheme Based on Elliptic Curve Cryptography," *IEEE Access*, vol. 10, pp. 34221–34227, 2022, doi: 10.1109/ACCESS.2022.3162231.
- [12] A. Abdaoui, A. Erbad, A. K. Al-Ali, A. Mohamed, and M. Guizani, "Fuzzy Elliptic Curve Cryptography for Authentication in Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 12, pp. 9987–9998, 2022, doi: 10.1109/JIOT.2021.3121350.
- [13] I. Ali, Y. Chen, C. Pan, and A. Zhou, "ECCHSC: Computationally and Bandwidth Efficient ECC-Based Hybrid Signcryption Protocol for Secure Heterogeneous Vehicle-to-Infrastructure Communications," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4435–4450, 2022, doi: 10.1109/JIOT.2021.3104010.
- [14] Purohit, Neha, and Shubhalaxmi Joshi. "Comprehensive Analysis on Various Cryptographic Algorithm for Constrained Devices." 2024 International Conference on Innovations and Challenges in Emerging Technologies (ICICET). IEEE, 2024.
- [15] Purohit, Neha, et al. "Pragmatic analysis of ECC based security models from an empirical perspective." *Journal of Discrete Mathematical Sciences and Cryptography* 26.3 (2023): 739-758.